

# DOKUMENTASI & PANDUAN SOLID PRINCIPLES

## Studi Kasus Laravel: E-Commerce Checkout Engine

Dokumentasi komprehensif mengenai penerapan 5 Prinsip SOLID (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, dan Dependency Inversion) dalam arsitektur modern Laravel. Disusun berdasarkan mini project **E-Commerce Checkout Engine** lengkap dengan skenario, kode, dan unit testing.

**Dipraktekkan Oleh:** Haris & Antigravity AI

**Framework:** Laravel 12 (PHP 8.2+)

**Status Test:** 100% PASS (10 Tests, 20 Assertions)

**Tanggal:** September 2026

## 1. Pendahuluan & Konsep Dasar

Prinsip **SOLID** adalah 5 panduan desain arsitektur berorientasi objek (OOP) yang bertujuan untuk membuat perangkat lunak menjadi lebih **modular**, **mudah dipelihara (maintainable)**, **mudah diuji (testable)**, dan **fleksibel untuk dikembangkan (scalable)**.

| Prinsip                      | Singkatan | Fokus Utama dalam Laravel                                                                    |
|------------------------------|-----------|----------------------------------------------------------------------------------------------|
| <b>Single Responsibility</b> | SRP       | Memisahkan Controller, Request Validation, Action, & Service Class.                          |
| <b>Open/Closed</b>           | OCP       | Menggunakan Strategy Pattern (Interface) agar fitur baru bisa ditambah tanpa ubah kode lama. |
| <b>Liskov Substitution</b>   | LSP       | Menjamin semua subclass/implementasi interface punya return type & behavior yang identik.    |
| <b>Interface Segregation</b> | ISP       | Memecah interface raksasa menjadi interface kecil & spesifik sesuai kebutuhan.               |
| <b>Dependency Inversion</b>  | DIP       | Bergantung pada abstraksi (Interface) via Constructor Injection & Service Container.         |

## 2. Struktur Folder & Arsitektur Project

Mini project **E-Commerce Checkout Engine** disusun di dalam direktori `app/` dengan pembagian layer yang jernih:

```

app/
├── Actions/
│   └── CheckoutAction.php # [SRP & DIP] Orchestrator logika bisnis checkout
├── Contracts/
│   ├── DiscountStrategyInterface.php # [OCP] Interface strategi diskon
│   ├── PaymentGatewayInterface.php # [DIP & LSP] Interface gateway pembayaran
│   ├── EmailNotifierInterface.php # [ISP] Interface notifikasi Email
│   └── SmsNotifierInterface.php # [ISP] Interface notifikasi SMS
├── Services/
│   ├── Discounts/
│   │   ├── DiscountCalculator.php # [SRP & OCP] Kalkulator diskon utama
│   │   ├── FixedDiscount.php # Implementasi diskon nominal
│   │   ├── PercentageDiscount.php # Implementasi diskon persentase
│   │   └── BuyOneGetOneDiscount.php # Implementasi diskon BOGO (Fitur Baru OCP)
│   ├── Payments/
│   │   ├── BankTransferPayment.php # Implementasi bayar Bank (LSP)
│   │   ├── EWalletPayment.php # Implementasi bayar E-Wallet (LSP)
│   │   └── CreditCardPayment.php # Implementasi bayar Kartu Kredit (LSP)
│   └── Notifications/
│       ├── EmailNotifier.php # Implementasi Notifier Email
│       └── SmsNotifier.php # Implementasi Notifier SMS
├── Http/
│   ├── Controllers/
│   │   └── CheckoutController.php # [SRP] Memproses HTTP Request & JSON Response
│   ├── Requests/
│   │   └── CheckoutRequest.php # [SRP] Validasi payload input
├── Providers/
│   └── AppServiceProvider.php # [DIP] Binding interface ke konkrit di Service Container

```

## 3. Single Responsibility Principle (SRP)

**Definisi: Satu kelas hanya boleh memiliki satu alasan untuk berubah.**

Dalam Laravel, kesalahan umum adalah menumpuk semua logika di *Controller* (validasi, kalkulasi, simpan DB, bayar, kirim email). Dengan SRP, kita membaginya ke dalam 3 layer utama:

- **CheckoutRequest:** Khusus menangani aturan validasi payload HTTP.
- **CheckoutAction:** Khusus mengorkestrasikan alur logika bisnis checkout.
- **CheckoutController:** Khusus menerima Request HTTP dan mengembalikan Response JSON.

### Contoh Kode CheckoutController.php:

```
<?php
namespace App\Http\Controllers;
use App\Actions\CheckoutAction;
use App\Http\Requests\CheckoutRequest;
use Illuminate\Http\JsonResponse;
class CheckoutController extends Controller
{
    public function __invoke(CheckoutRequest $request, CheckoutAction $action): JsonResponse
    {
        $result = $action->execute($request->validated());
        return response()->json([
            'status' => 'success',
            'message' => 'Checkout berhasil diproses',
            'data' => $result
        ], 201);
    }
}
```

## 4. Open/Closed Principle (OCP)

**Definisi: Terbuka untuk ekstensi (fitur baru), tertutup untuk modifikasi (kode lama).**

Sistem kalkulasi diskon dirancang menggunakan **Strategy Pattern** via `DiscountStrategyInterface`. Ketika ada aturan diskon baru seperti `BuyOneGetOneDiscount`, kita **cukup membuat class baru** tanpa mengedit 1 baris pun dari class `DiscountCalculator.php`!

## Interface & Strategy Class:

```
// DiscountStrategyInterface.php
interface DiscountStrategyInterface
{
    public function apply(float $totalAmount): float;
    public function name(): string;
}
// DiscountCalculator.php (Tak perlu diubah saat ada diskon baru!)
class DiscountCalculator
{
    public function calculate(float $totalAmount, ?DiscountStrategyInterface $strategy = null): float
    {
        if (!$strategy) return $totalAmount;
        return $strategy->apply($totalAmount);
    }
}
```

## 5. Liskov Substitution Principle (LSP)

**Definisi: Subclass/Implementasi harus dapat menggantikan superclass/interface tanpa merusak aplikasi.**

Semua gateway pembayaran (BankTransferPayment, EWalletPayment, dan CreditCardPayment) diwajibkan mengimplementasikan PaymentGatewayInterface dan **mengembalikan struktur array yang identik**.

### Kontrak Konsisten (PaymentGatewayInterface & CreditCardPayment):

```
// PaymentGatewayInterface.php
interface PaymentGatewayInterface
{
    public function charge(float $amount, array $paymentDetails): array;
}
// CreditCardPayment.php (Wajib mengembalikan tipe & struktur array yang konsisten)
class CreditCardPayment implements PaymentGatewayInterface
{
    public function charge(float $amount, array $paymentDetails): array
    {
        return [
            'success' => true,
            'transaction_id' => 'TRX-CC-' . uniqid(),
            'message' => "Pembayaran via Kartu Kredit berhasil."
        ];
    }
}
```

## 6. Interface Segregation Principle (ISP)

**Definisi: Klien tidak boleh dipaksa bergantung pada method yang tidak mereka gunakan.**

Daripada membuat interface gemuk NotifierInterface yang berisi sendEmail(), sendSms(), sendWhatsapp() sekaligus, kita pecah menjadi interface yang ramping: EmailNotifierInterface dan SmsNotifierInterface.

```
interface EmailNotifierInterface
{
    public function sendEmail(string $email, string $message): bool;
}
interface SmsNotifierInterface
{
    public function sendSms(string $phoneNumber, string $message): bool;
}
```

## 7. Dependency Inversion Principle (DIP)

**Definisi: High-level module tidak boleh bergantung pada Low-level module. Keduanya harus bergantung pada abstraksi.**

CheckoutAction bergantung pada abstraksi interface (EmailNotifierInterface, SmsNotifierInterface). Laravel Service Container di AppServiceProvider.php secara otomatis menyuntikkan (dependency injection) instance konkritnya.

```
// AppServiceProvider.php
public function register(): void
{
    $this->app->bind(EmailNotifierInterface::class, EmailNotifier::class);
    $this->app->bind(SmsNotifierInterface::class, SmsNotifier::class);
}
```

## 8. Pengujian Automated (Unit & Feature Test)

Kode yang dirancang dengan SOLID sangat **testable** (mudah diuji). Pengujian dapat dilakukan tanpa tergantung pada Service External asli.

```
===== PENGUJIAN LARAVEL PHPUNIT =====
PASS Tests\Unit\DiscountTest
✓ can calculate fixed discount
✓ calculate percentage discount
✓ buyone getone
✓ returns original amount when no discount
PASS Tests\Unit\PaymentGatewayTest
✓ bank transfer payment conforms to contract
✓ ewallet payment conforms to contract
✓ credit card payment conforms to contract
PASS Tests\Feature\CheckoutTest
✓ can checkout successfully with discount and ewallet
Tests: 10 passed (20 assertions)
Duration: 0.30s
=====
```

## 9. Cheatsheet & Matriks Ringkasan SOLID

| Prinsip    | Pertanyaan Kunci Saat Coding                                           | Solusi Arsitektur Laravel                                  |
|------------|------------------------------------------------------------------------|------------------------------------------------------------|
| <b>SRP</b> | Apakah class ini punya lebih dari 1 alasan untuk berubah?              | Gunakan Form Request, Action Class, & Service Class.       |
| <b>OCP</b> | Apakah saya harus edit kode lama jika ada variasi fitur baru?          | Gunakan Strategy Pattern (Interface & Class Khusus).       |
| <b>LSP</b> | Apakah semua implementasi interface punya return type & behavior sama? | Type-hint return type di interface secara ketat.           |
| <b>ISP</b> | Apakah ada method di interface yang tidak dipakai oleh class anak?     | Pecah interface raksasa menjadi interface-interface tipis. |
| <b>DIP</b> | Apakah class saya memanggil new ConcreteClass() secara keras?          | Inject Interface di __construct & bind di ServiceProvider. |